

03 Matrix Multiplication

3.1 The First View of Matrix Multiplication

Row-Column Alignment, Inner-Product Expansion, and Vector Inner/Outer Products



Linear Algebra Made Easy and Visual | Open-source code & drafts: <https://github.com/Visualize-ML>

Core Skills for This Section

1

Row-column alignment for shape matching: for $\mathbf{AB}$, line up $\mathbf{A}$'s rows on the left and $\mathbf{B}$'s columns on top to instantly see whether the product is valid and what shape it has

3

Vector inner products can be written as matrix multiplication: a 1×1 matrix is just a scalar

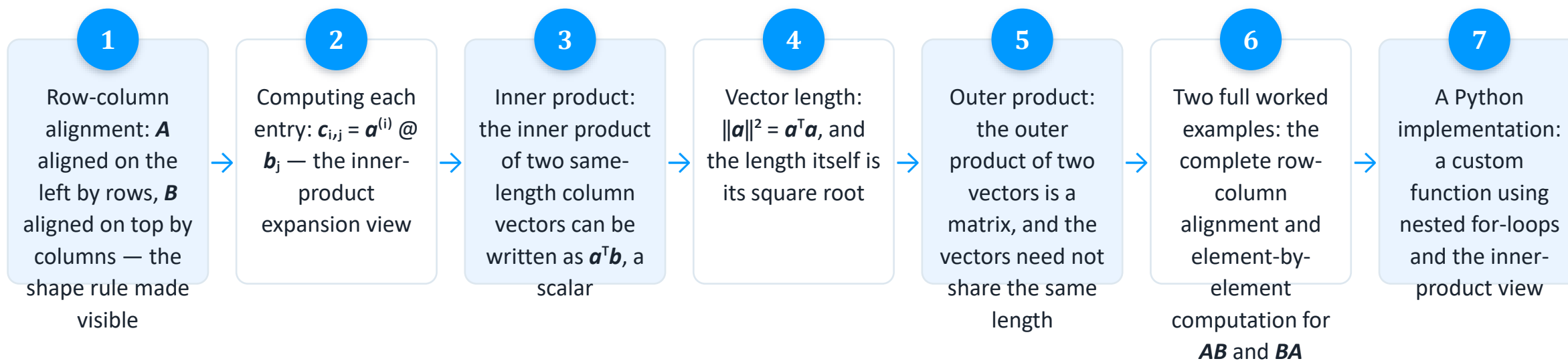
2

The inner-product view of each entry: every entry of the product is the inner product of a row of $\mathbf{A}$ and a column of $\mathbf{B}$

4

Vector outer products generate matrices: constructing rank-one matrices, setting up the next section's “second view of matrix multiplication”

How This Section Unfolds



A New Way to Look at Matrix Multiplication

The previous chapter already introduced matrix multiplication; this chapter is devoted entirely to different ways of understanding it

- The first view of matrix multiplication aligns matrices **A** and **B** in a specific way, making the shape requirements of the product visible at a glance
- This view also reveals exactly how each entry of the result is computed — through the inner product

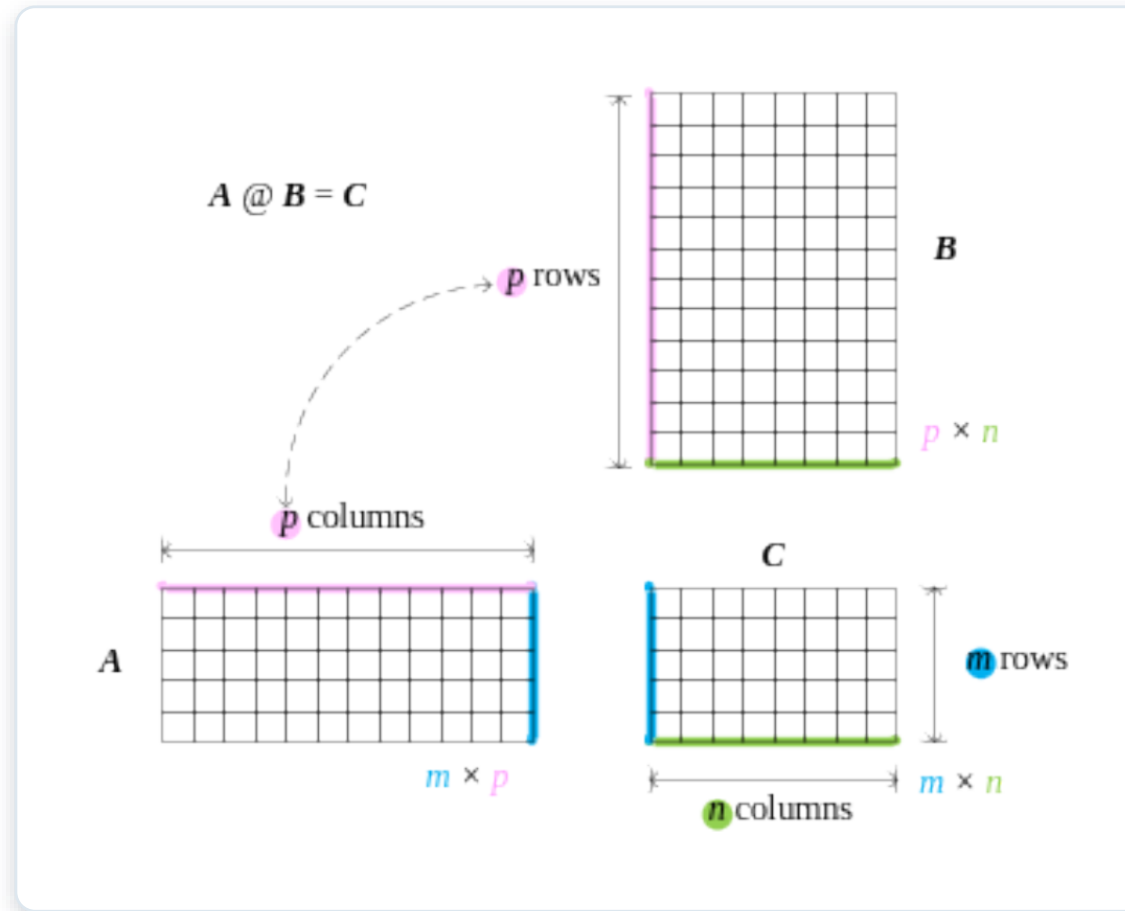
Row and Column Alignment

Matrix A is $m \times p$, matrix B is $p \times n$; place A to the left of result C , row-aligned, and B above C , column-aligned

- Placing A to the left of C means A 's row count m determines C 's row count m
- Placing B above C means B 's column count n determines C 's column count n ; so $A @ B$ has shape $m \times n$

Figure 1 · A Aligned to the Left of AB by Rows; B Aligned Above AB by Columns

- This alignment reveals matrix multiplication's core rule: **A**'s column count must equal **B**'s row count, or the two simply cannot line up
- **A** is $m \times p$, **B** is $p \times n$; the shared dimension p “cancels out,” leaving **C** with shape $m \times n$



Computing the (i, j) Entry of AB

Once aligned, each entry $c_{i,j}$ of C is the sum of the products of corresponding entries in A 's row i and B 's column j

- This spatial-alignment view not only clarifies the shape of the product but gives an intuitive sense of how the two matrices' information is combined
- Next, let's see exactly how the (i, j) entry gets computed, step by step

Figure 2 · Computing the (i, j) Entry of AB

- A 's row i (light-to-dark blue gradient) lines up with B 's column j (light-to-dark orange gradient); where they meet is exactly $c_{i,j}$
- A sits to the left of AB , row-aligned; B sits above AB , column-aligned

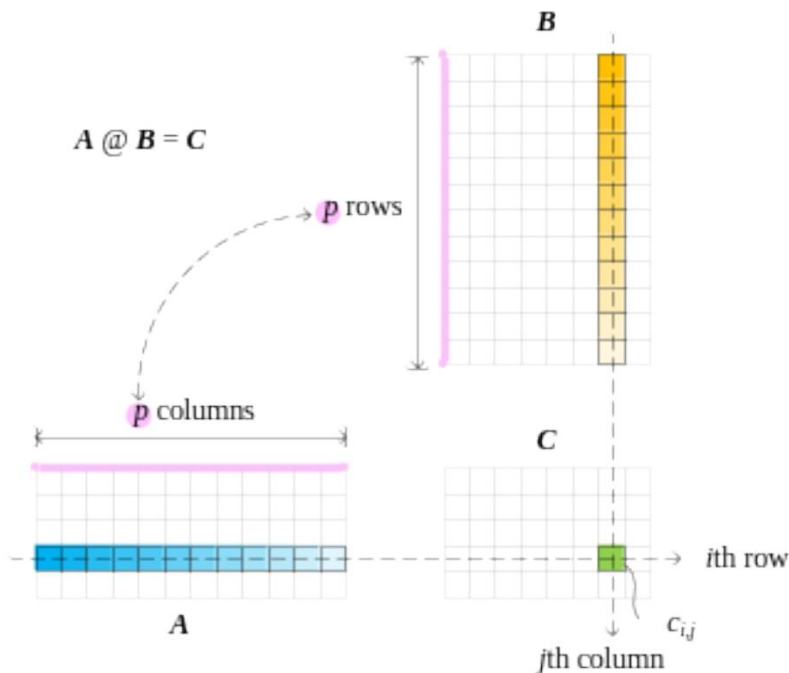


图 2. 计算 AB 的第 i 行、第 j 列元素： A 放在 AB 的左侧，行对齐； B 放在 AB 的上面，列对齐

The First View Is Also Called Inner-Product Expansion

$$c_{ij} = \mathbf{a}^{(i)} @ \mathbf{b}_j = \mathbf{a}^{(i)T} \cdot \mathbf{b}_j$$

- Write matrix $\mathbf{A}$ as a set of row vectors $\mathbf{a}^{(i)}$, and matrix $\mathbf{B}$ as a set of column vectors $\mathbf{b}_j$
- The (i, j) entry of $\mathbf{C} = \mathbf{AB}$ can be written either as a matrix product or as a vector inner product

Figure 3 · The Inner-Product Expansion View of $\mathbf{a}^{(i)}$ @ $\mathbf{b}_j = c_{i,j}$

- Row vector $\mathbf{a}^{(i)}$ has p entries, column vector $\mathbf{b}_j$ has p entries; multiplying corresponding entries and summing gives scalar $c_{i,j}$
- This is exactly the linear-algebra inner product — and the core operation behind Python's `numpy.dot()`

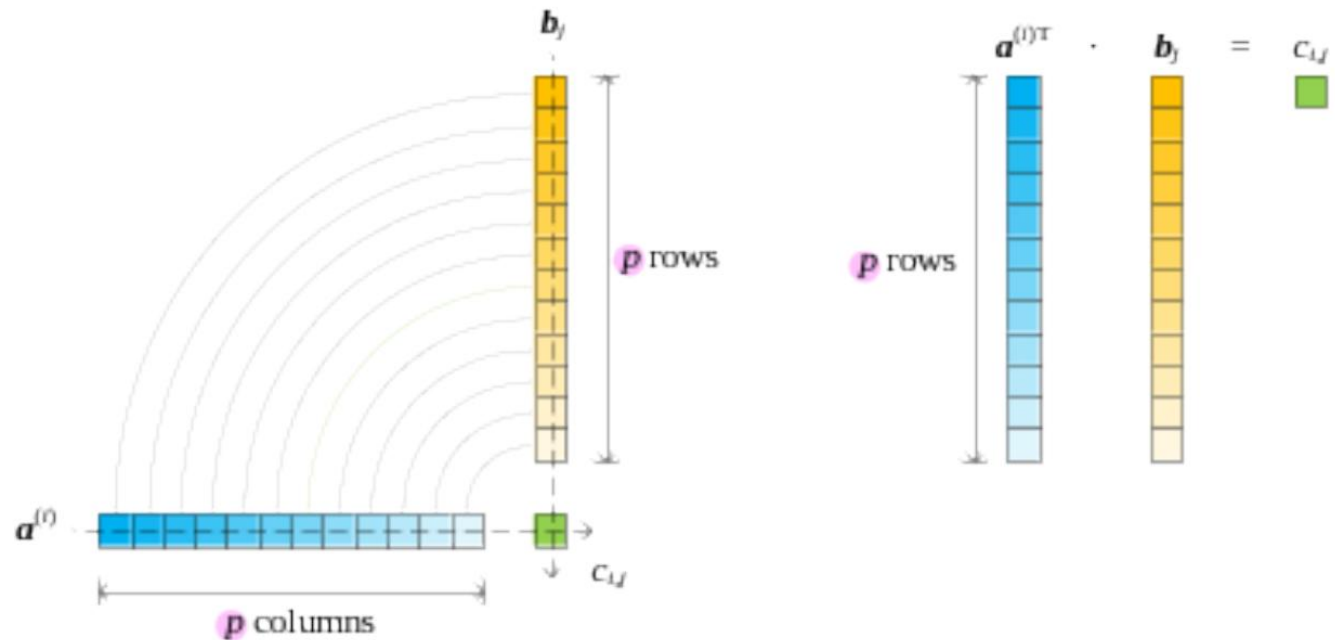


图 3. 内积展开视角看矩阵乘积 $\mathbf{a}^{(i)}$ @ $\mathbf{b}_j = c_{i,j}$

The Inner Product Can Also Be Written as Matrix Multiplication

For n -dimensional vectors $\mathbf{a}$, $\mathbf{b}$: $\mathbf{a} \cdot \mathbf{b} = \langle \mathbf{a}, \mathbf{b} \rangle = \mathbf{a}^T \mathbf{b} = \mathbf{b}^T \mathbf{a}$

- The inner product of two same-length column vectors can be written with dot notation, or as matrix multiplication $\mathbf{a}^T @ \mathbf{b}$ or $\mathbf{b}^T @ \mathbf{a}$
- By shape: $(1 \times n) @ (n \times 1)$ has the shared dimension n “cancel out,” leaving a (1×1) result — a scalar

Figure 4 · The Vector Inner Product Can Also Be Written as Matrix Multiplication

- Given $\mathbf{a} = [1, 2, 3]^T$ and $\mathbf{b} = [4, 5, 6]^T$: $\mathbf{a}^T @ \mathbf{b} = [1 \ 2 \ 3] @ [4, 5, 6]^T = 32$, and $\mathbf{b}^T @ \mathbf{a}$ also equals 32
- A 1×1 matrix can be treated as a scalar; whether you compute $\mathbf{a}^T \mathbf{b}$ or $\mathbf{b}^T \mathbf{a}$, the result is identical

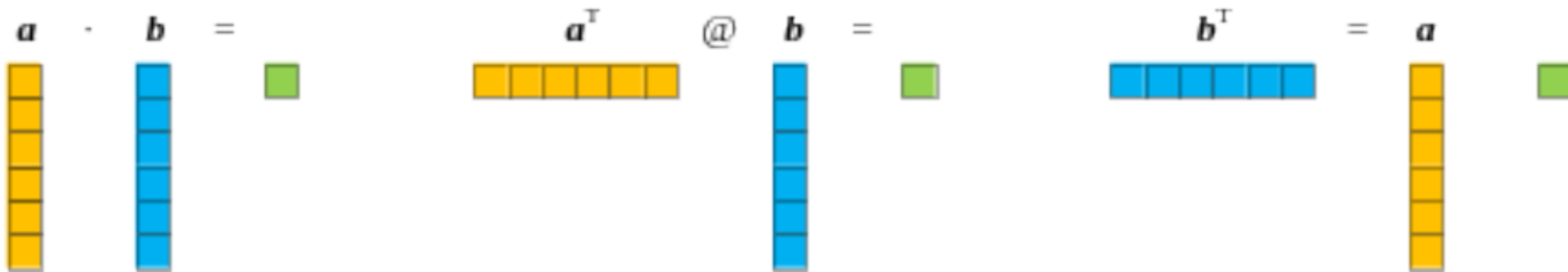


图 4. 向量内积也可以写成矩阵乘法形式

Vector Length Can Also Be Computed by Matrix Multiplication

$$\|\mathbf{a}\|^2 = \|\mathbf{a}\|_2^2 = \mathbf{a} \cdot \mathbf{a} = \langle \mathbf{a}, \mathbf{a} \rangle = \mathbf{a}^T \mathbf{a}, \text{ so } \|\mathbf{a}\| = \sqrt{\mathbf{a}^T \mathbf{a}}$$

- The square of the length (magnitude, L2 norm, or Euclidean norm) of column vector $\mathbf{a}$ can be found via the inner product or via matrix multiplication
- Given $\mathbf{a} = [1, 2, 3]^T$: $\|\mathbf{a}\| = \sqrt{[1 \ 2 \ 3] @ [1, 2, 3]^T} = \sqrt{14}$

Figure 5 · The Squared Vector Length Can Also Be Written as Matrix Multiplication

- $\mathbf{a} \cdot \mathbf{a}$ and $\mathbf{a}^T @ \mathbf{a}$ give exactly the same result: the sum of the squares of $\mathbf{a}$'s components
- This relationship resurfaces again and again in later topics — vector spaces, projections, least squares

The diagram illustrates two equivalent ways to calculate the squared length of a vector $\mathbf{a}$. On the left, the dot product $\mathbf{a} \cdot \mathbf{a}$ is shown, where two vertical yellow bars representing the vector $\mathbf{a}$ are separated by a dot, followed by an equals sign and a green square representing the result. On the right, the matrix multiplication $\mathbf{a}^T @ \mathbf{a}$ is shown, where a horizontal yellow bar representing the transpose $\mathbf{a}^T$ is followed by an '@' symbol, then a vertical yellow bar representing $\mathbf{a}$, followed by an equals sign and a green square representing the result.

图 5. 向量长度的平方也可以写成矩阵乘法形式

The Outer Product: The Opposite of the Inner Product

The inner product of two vectors is a scalar; the outer product (outer product) of two vectors is a matrix

- Outer product $\mathbf{a} @ \mathbf{b}^T$ does not require $\mathbf{a}$ and $\mathbf{b}$ to have the same length: $\mathbf{a}$ is m-dimensional, $\mathbf{b}$ is n-dimensional, and the result is an $m \times n$ matrix
- This special form of matrix multiplication sets up the next section's “second view of matrix multiplication” and rank-one matrices

Figure 6 · The Two Outer Products of Vectors a and b

- $a @ b^T$ multiplies each entry of a by all of b , “coloring in” the matrix row by row; $b @ a^T$ does the reverse
- The two outer products are transposes of each other: $(a @ b^T)^T = b @ a^T$

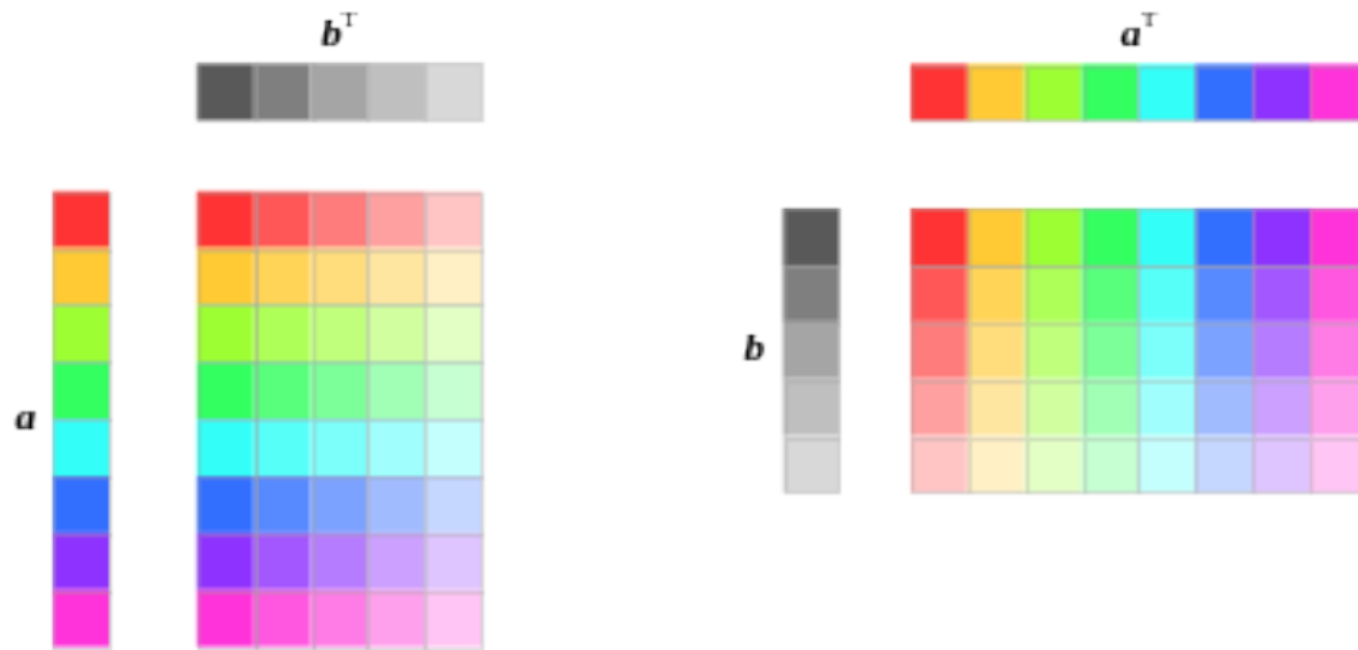


图 6. 向量 a 、 b 的两个外积

An Outer-Product Example: Shapes Can Differ

Given $\mathbf{a} = [1,2,3,4,5]^T$ and $\mathbf{b} = [1,2,3]^T$ (not required to have the same length)

- $\mathbf{a} @ \mathbf{b}^T$: $(5 \times 1) @ (1 \times 3)$ — the shared dimension 1 “cancels out,” leaving a 5×3 result
- $\mathbf{b} @ \mathbf{a}^T$: $(3 \times 1) @ (1 \times 5)$ — the shared dimension 1 “cancels out,” leaving a 3×5 result; the two results are transposes of each other

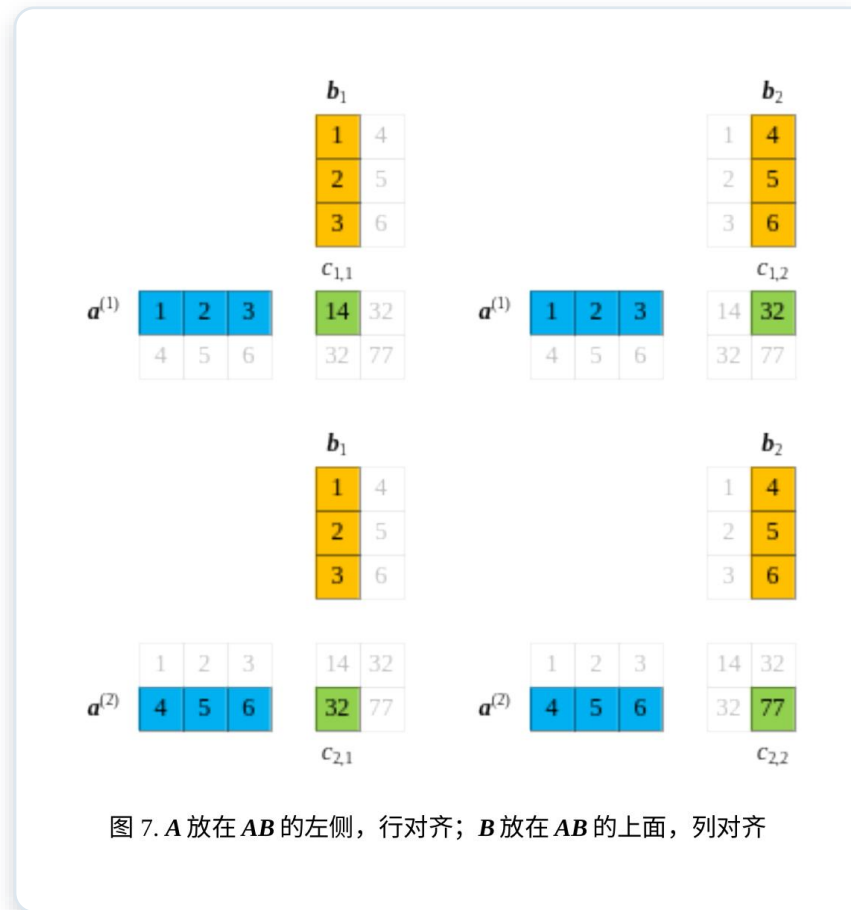
Back to an Earlier Example: $A @ B$

$$A = [1,2,3; 4,5,6], B = [1,4; 2,5; 3,6], A @ B = [14,32; 32,77]$$

- Write A as two row vectors $\mathbf{a}^{(1)}, \mathbf{a}^{(2)}$ and B as two column vectors $\mathbf{b}_1, \mathbf{b}_2$; then $A @ B$ becomes four inner products
- $c_{1,1} = \mathbf{a}^{(1)} @ \mathbf{b}_1, c_{1,2} = \mathbf{a}^{(1)} @ \mathbf{b}_2, c_{2,1} = \mathbf{a}^{(2)} @ \mathbf{b}_1, c_{2,2} = \mathbf{a}^{(2)} @ \mathbf{b}_2$

Figure 7 · **A** Aligned to the Left of **AB** by Rows; **B** Aligned Above **AB** by Columns

- $\mathbf{a}^{(1)}$ @ $\mathbf{b}_1$ multiplies a 1×3 row vector by a 3×1 column vector, giving a 1×1 matrix — effectively a scalar
- Filling in the four inner products $c_{1,1}$, $c_{1,2}$, $c_{2,1}$, $c_{2,2}$ at their positions gives exactly $\mathbf{A} @ \mathbf{B} = [14, 32; 32, 77]$



Now the Other Order: $B @ A$

$$\mathbf{D} = \mathbf{B} @ \mathbf{A} = [1,4; 2,5; 3,6] @ [1,2,3; 4,5,6] = [17,22,27; 22,29,36; 27,36,45]$$

- Write $\mathbf{B}$ as three row vectors $\mathbf{b}^{(1)}$, $\mathbf{b}^{(2)}$, $\mathbf{b}^{(3)}$ and $\mathbf{A}$ as three column vectors $\mathbf{a}_1$, $\mathbf{a}_2$, $\mathbf{a}_3$; then $\mathbf{B} @ \mathbf{A}$ becomes nine inner products
- Same pair of matrices, opposite order — the result isn't just different, even the shape of the result changes, from 2×2 to 3×3

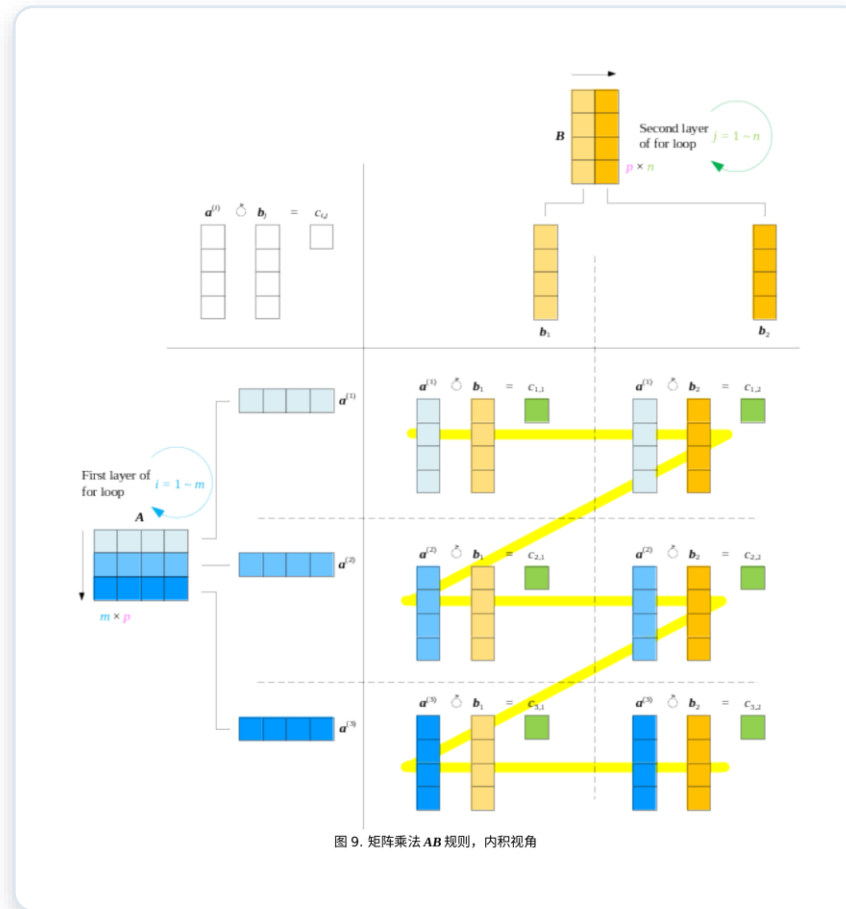
Figure 8 · Computing Each Entry: B Aligned to the Left of BA by Rows; A Aligned Above BA by Columns

- $d_{1,1}, d_{1,2}, \dots, d_{3,3}$ — nine inner products in total, each pairing a $b^{(i)}$ with an a_j
- Comparing Figures 7 and 8, the first view of matrix multiplication applies to A, B of any compatible shape, not just square matrices



Figure 9 · The Rule for AB, the Inner-Product View

- The outer loop walks through each row of $\mathbf{A}$ ($i = 1 \sim m$); the inner loop walks through each column of $\mathbf{B}$ ($j = 1 \sim n$)
- Each time $\mathbf{a}^{(i)} @ \mathbf{b}_j$ is computed, the result fills row i , column j of $\mathbf{C}$ — until the entire $m \times n$ matrix is filled



Code 1 · A Custom Python Function for Matrix Multiplication

LA_03_01_02.ipynb

- `def matrix_multiplication_inner(A, B)` first reads the shapes of A and B, and raises an error if `p_A` doesn't equal `p_B`
- It initializes result matrix C as an (m, n) matrix of zeros, then loops over every row i and every column j
- For each (i, j), it takes `row_i_A` (row i of A) and `col_j_B` (column j of B), computes their inner product with `numpy.dot()`, and assigns it to `C[i, j]`

```
## 初始化
import numpy as np

## 自定义函数，矩阵乘法第一视角
def matrix_multiplication_inner(A, B):

    # 获取矩阵 A 和 B 的形状
    m, p_A = A.shape
    p_B, n = B.shape

    # 检测矩阵形状是否符合矩阵乘法规则
    if p_A != p_B:
        raise ValueError('Dimensions do not match')

    # 初始化结果矩阵 C，形状 (m, n)，初始值设为 0
    C = np.zeros((m, n))

    # 计算每个元素
    for i in range(m):
        for j in range(n):
            row_i_A = A[i, :]          # A 的第 i 行 (行向量)
            col_j_B = B[:, j]          # B 的第 j 列 (列向量)
            C[i, j] = np.dot(row_i_A, col_j_B) # 计算内积

    return C

## 矩阵乘法
A = np.array([[1, 2, 3],
              [4, 5, 6]])
B = A.T

## 矩阵乘法
matrix_multiplication_inner(A, B)
matrix_multiplication_inner(B, A)
```

Inner-Product Expansion: The First Key to Matrix Multiplication

Align rows and columns to see the shape; expand into inner products to compute each entry — this is the most intuitive first view of matrix multiplication

- The inner product folds two vectors into a scalar; the outer product spreads two vectors out into a matrix — the two are dual to each other
- The next section starts from the outer product to introduce the second view of matrix multiplication: understanding AB as a sum of rank-one matrices

Key Terms in This Section

Row-Column Alignment *A aligned to the left by rows, B aligned above by columns, revealing the shape of AB*

Inner Product $a \cdot b = a^T b = b^T a$ — the product of two same-length vectors, a scalar

Rank-One Matrix *A matrix generated by a vector outer product — covered in depth in the next section*

The Shape Rule *A @ B is valid only when A's column count equals B's row count*

Row / Column Vectors *A written as a set of row vectors $a(i)$; B written as a set of column vectors b_j*

Inner-Product Expansion *The first view of matrix multiplication: $c_{i,j} = a(i) @ b_j$, computed entry by entry*

Outer Product $a @ b^T$ — the product of two vectors, a matrix; the vectors need not share a length

Vector Length $\|a\| = \sqrt{a^T a}$, also called the L2 norm, magnitude, or Euclidean norm

Scalar *A 1×1 matrix can be treated as a scalar*

Nested For-Loop *Outer loop over row index i , inner loop over column index j , computing $C[i,j]$ one at a time*

GO FURTHER

Companion Code and Further Reading

Further Reading · LA_03_01_01.ipynb

Computes every worked example in this section using the first view of matrix multiplication — work through it line by line

Further Reading · LA_03_01_02.ipynb

A custom Python function for matrix multiplication, implemented from the inner-product view

Related Content · The Second View of Matrix Multiplication

The next section starts from the outer product and understands AB as a sum of rank-one matrices

Open-Source Repository

Code and figures for the whole book: <https://github.com/Visualize-ML>

Questions Q1 – Q9

Q1 Given column vector $a=[1,2]^T$, use the first view of matrix multiplication to lay out and compute $a^T @ a$ and $a @ a^T$.

Q3 Given $A=[1,2;3,4]$ and $b=[1,2]^T$, use the first view of matrix multiplication to lay out and compute $A @ b$ and $b^T @ A^T$.

Q5 Modify Code 1 by swapping the inner and outer for-loops, and check whether the result changes.

Q7 Write a function `outer_product_manual(a, b)` that computes $a @ b^T$ with nested for-loops (without calling `np.outer`), and compare it against `np.outer(a, b)`.

Q9 For several randomly generated vectors, check whether `np.linalg.norm(a)` matches `np.sqrt(a.T @ a)`.

Q2 Given column vectors $a=[1,2]^T$ and $b=[1,2,3]^T$, use the first view of matrix multiplication to lay out and compute $a @ b^T$ and $b @ a^T$.

Q4 Given $A=[1,2;3,4;5,6]$, use the first view of matrix multiplication to lay out and compute $A @ A^T$ and $A^T @ A$.

Q6 Write a function `inner_product_via_matmul(a, b)` that computes the vector inner product using $a.T @ b$, and compare it against `np.dot(a, b)`.

Q8 Using Python, reproduce Figure 7: given the A and B from the first example, print out each of the four inner products $a(1)@b1$, $a(1)@b2$, $a(2)@b1$, $a(2)@b2$ step by step.

Questions Q10 – Q18

Q10 Write a function `check_shape_match(A, B)` that checks whether A's column count equals B's row count before multiplying, mirroring the shape check in Code 1.

Q12 Construct a 5-dimensional column vector `a` and a 3-dimensional column vector `b`, compute `a @ bT` and `b @ aT` in Python, and compare the resulting shapes against this section's conclusions.

Q14 Using the `time` module, compare the runtime of `matrix_multiplication_inner(A, B)` against NumPy's built-in `A @ B` on large random matrices (e.g., 200×200).

Q16 Given three same-length vectors `a`, `b`, `c`, check whether `(aT@b)*c` equals `a*(bT@c)`, and explain the result from a shape-analysis perspective.

Q18 Check whether `D = B @ A` from the second example satisfies `D = DT` (symmetric), and explain why `BA` is always symmetric when `B = AT`.

Q11 Given `A=[1,2,3;4,5,6]` and `B=AT`, compute `A @ B` and `B @ A` separately, and explain why the two results have different shapes.

Q13 Write a function `rank_one_check(M)` that determines whether every row of a given matrix `M` is a scalar multiple of the same row vector (i.e., whether `M` could be an outer product).

Q15 Modify Code 1 to implement matrix multiplication using pure vectorized NumPy operations (no explicit for-loops), and verify the result matches.

Q17 Write a function `visualize_alignment(A, B)` that prints the shapes of `A`, `B`, and `C` in the style of Figure 1 (`m×p`, `p×n`, `m×n`).

Questions Q19 – Q25

Q19 For a random 4×2 matrix A , compute $A @ A^T$ and $A^T @ A$, and check whether both results are symmetric matrices.

Q21 Using `matrix_multiplication_inner`, verify that swapping the inner and outer loops from Q5 gives the exact same result matrix, only in a different computation order.

Q23 Write a function `count_multiplications(A, B)` that returns the total number of scalar multiplications $m \times n \times p$ needed for $A @ B$, and check it against the actual operation count in your nested-loop implementation.

Q25 Combining what you learned in this section, write a small program that takes any two matrices, uses row-column alignment (Figure-1 style) to check compatibility, and if compatible, computes and prints the product using the inner-product view.

Q20 Write a function that compares `trace(A @ AT)` against the sum of the squares of all of A 's entries (the squared Frobenius norm), and check whether they're equal.

Q22 Construct a 3×1 column vector a and a 1×3 row vector b^T , compute $a @ b^T$, and use `np.linalg.matrix_rank()` to confirm the result has rank 1.

Q24 In NumPy, try multiplying two matrices with incompatible shapes (e.g., 2×3 and 2×3), capture and print the resulting error, and compare it to the `ValueError` raised in Code 1.

10 Prompts to Explore Further (1/2)

1

Explain why matrix multiplication $A @ B$ requires A 's column count to equal B 's row count, using row-column alignment to convey the spatial intuition.

2

Prove that entry $c_{i,j}$ of product $C = AB$ equals the inner product of A 's row i and B 's column j .

3

Explain why the inner products $a^T b$ and $b^T a$ of two same-length vectors are equal, while the outer products ab^T and ba^T generally are not (except as transposes of each other).

4

Explain why vector length $\|a\|$ can be written as $\sqrt{a^T a}$, and how this connects to the Pythagorean theorem.

5

Explain why the outer product $a @ b^T$ doesn't require a and b to have the same dimension, while the inner product $a^T @ b$ does.

10 Prompts to Explore Further (2/2)

6

Explain why swapping the order of the nested for-loops in Code 1 (columns before rows) doesn't change the final result of the matrix multiplication.

7

Explain why the shape of $B @ A$ can be completely different from $A @ B$, and give an example of when the two shapes happen to match.

8

Explain why the matrix generated by an outer product $a @ b^T$ is always a “rank-one matrix,” and how this sets up the next section's second view of matrix multiplication.

9

Explain the relationship between the first view of matrix multiplication (inner-product expansion) and a vectorized NumPy implementation, and why vectorization is usually faster.

10

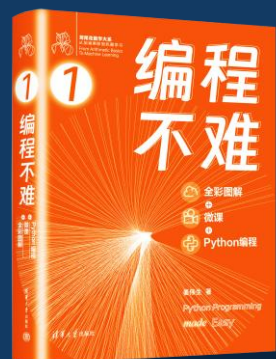
Give an example of how the inner-product view of matrix multiplication shows up in machine learning — for instance, how the forward pass of a fully connected layer computes each output via an inner product.

数学

数学基础

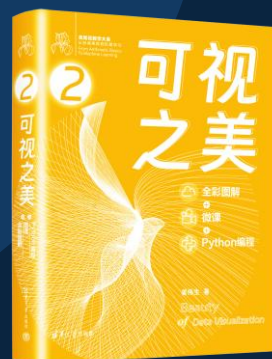
线性代数

概率统计

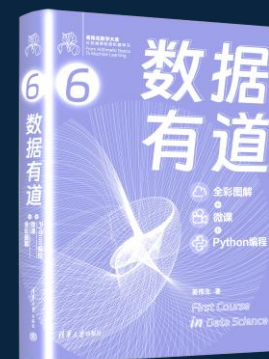
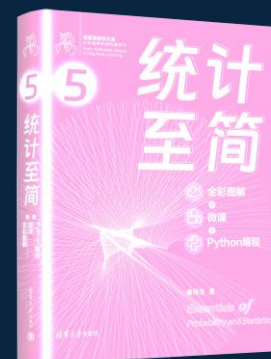
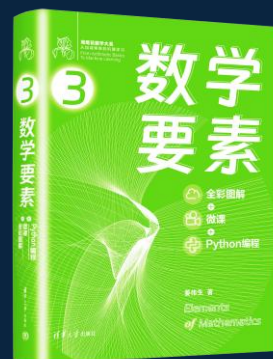


Python编程

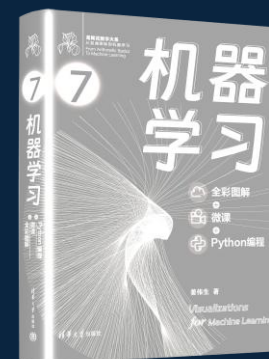
工具



数据可视化



数据分析
图和网络



回归、降维
分类、聚类

实践

